

Assignment 5: Lenia – An artificial life

Plabon Shaha and Guillem Masdemont

University of Ljubljana, Spring Semester, EMAI Cohort 25–27

May 26, 2026

Evaluation We use different versions to evaluate performance metrics.

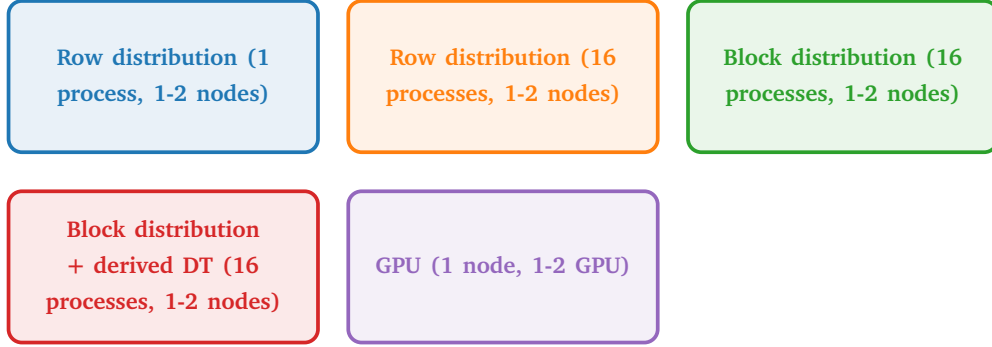


Table 1: Overview of Lenia Simulation Parallel Configurations

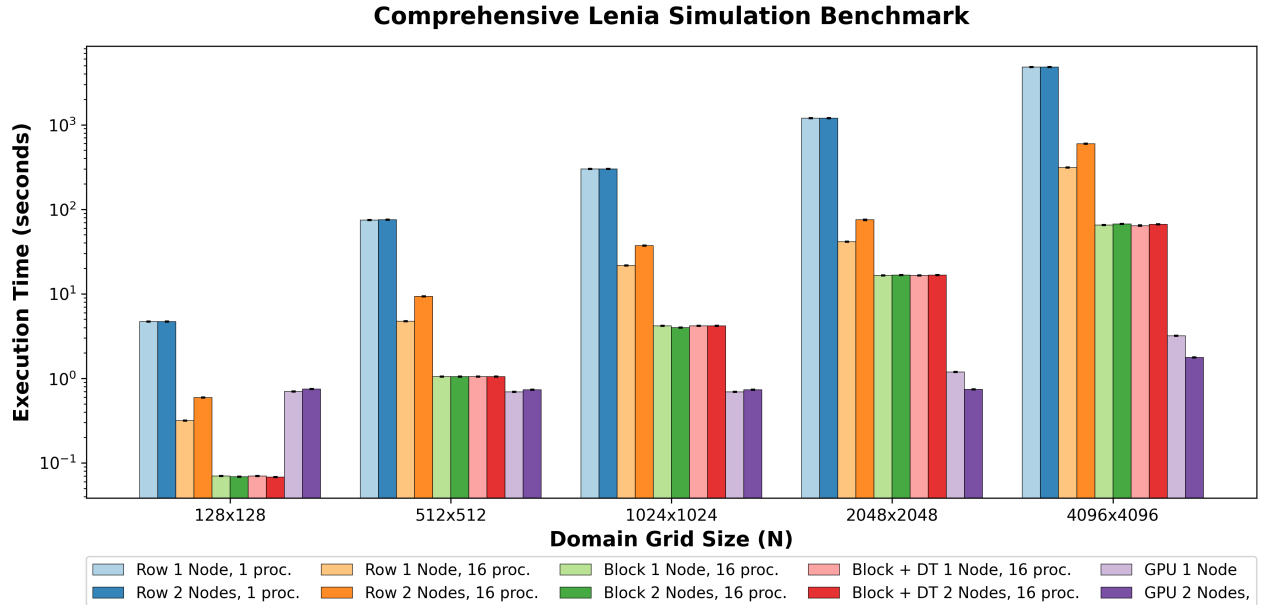


Figure 1: Performance comparison of the aforementioned optimization layers.

For Row distribution, splitting across 2 nodes slows down performance because full-width halo rows must travel over a slower network instead of fast intra-node shared memory. Block distribution outperforms row distribution because its 4×4 grid layout sends half as much communication data per process. Using MPI Derived Types (Block+DT) yields a negligible difference over manual packing because the creation overhead completely offsets any packing efficiency. Block distribution significantly reduces halo data, scaling to 2 nodes remains cheap enough to keep performance nearly identical to a single node. Finally, GPUs are heavily bottlenecked by launch and transfer overheads on small grids ($N = 128$), but their massive parallelism completely dominates large scales ($N = 4096$). A second GPU succeeds only after there is enough compute work to offset the split costs.

1. Now, we increase the width of the border area exchanged between processes so by performing additional computations, we can exchange data only every second, third, etc. iteration. We show the results for the 2048 pixel image and 4,16, 34 processes. The results are presented in the following image

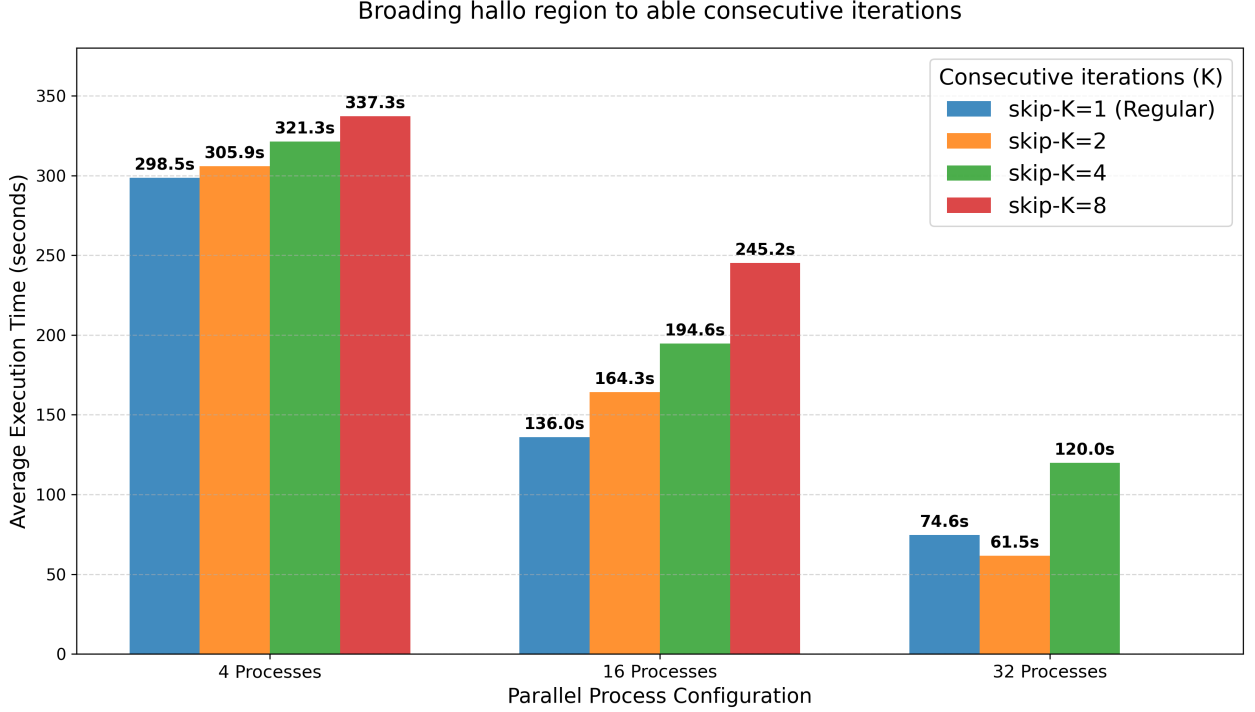


Figure 2: Performance comparison of the aforementioned optimization layers.

Widening the halo region by a factor of K trades a larger local computation burden for fewer but larger MPI exchanges. For the $K = 1$ baseline, communication is already cheap at high process counts (32 processes), leaving very little network overhead to amortize. Consequently, using higher K factors always hurts performance at 4 and 16 processes because calculating a wider border forces processes to perform massive redundant computations that their neighbors would have completed anyway. Since each process already owns a large chunk of the domain at lower process counts, this extra work quickly outweighs any communication savings. The only exception occurs at 32 processes with $K = 2$, where the local domain strips are so thin that communication overhead is high enough that skipping every other exchange actually saves time. However, setting $K = 4$ or $K = 8$ overcorrects even at this scale, causing redundant edge computations to dominate and drive runtimes back up.